

Libroware Masterclass

From Idea to Production — Build a Full-Stack Library Management System

A hands-on course in React, TypeScript, GraphQL, Prisma, PostgreSQL, Docker, and CI/CD

How to use this book

This is not a book you read once — it is a book you *type along with*. Every chapter that introduces a new piece of the system ends with an **Exercise** box. Do the exercise on your own machine, in your own folder, before moving to the next chapter. By the end of Part 10, you will have rebuilt — by hand, line by line — a working copy of Libroware: a real library management system with user accounts, book borrowing, fines, reservations, reviews, notifications, multi-language support, and a Docker-based deployment pipeline.

We assume **zero prior experience** with any of the specific tools used here. We do *not* assume you've never touched a computer — but we will explain what npm is, what a database migration is, what a JWT is, and why GraphQL exists, before we use any of them.

The course is split into 11 parts:

Part	Title	Depth
0	Orientation	Read
1	Foundations for True Beginners	Read
2	Project Setup & Monorepo Structure	Hands-on
3	Designing the Database (Prisma + PostgreSQL)	Hands-on
4	Building the GraphQL API	Hands-on, deep dive
5	Building the Frontend	Hands-on, deep dive
6	Internationalization (i18n)	Hands-on
7	Auth Hardening: Session Expiry	Hands-on, real-world bug fix
8	Cross-Cutting Concerns (survey)	Read + light exercises
9	Multi-Platform Builds (survey)	Read
10	Containerization & Deployment	Hands-on, deep dive
11	Capstone Checklist & Next Steps	Read

Part 0 — Orientation

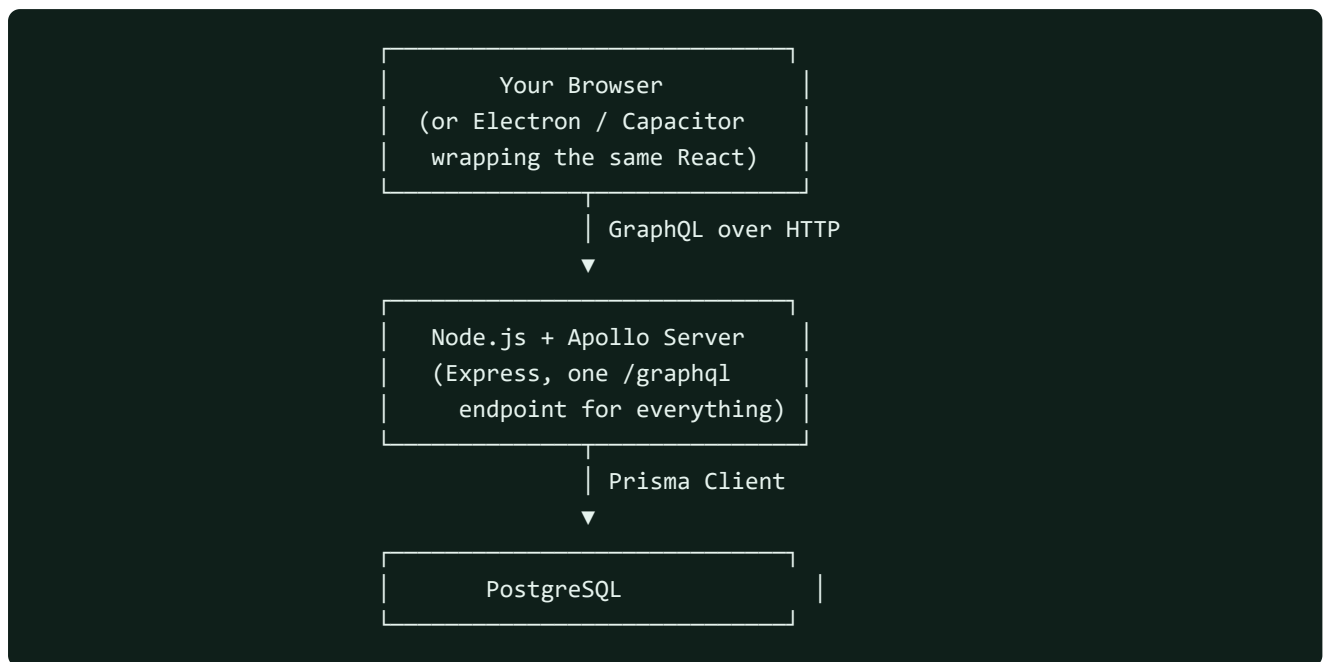
0.1 What we're building, and why

Libroware is a library management system. A library has books, and it has people who borrow books. That sentence alone hides almost every interesting problem in software engineering:

- **Who is allowed to do what?** A regular user can borrow a book. A librarian can add new books. An admin can create other librarians. This is *authentication* (who are you?) and *authorization* (what are you allowed to do?).
- **What happens when two people want the same book?** We need to track *availability*, and let people *reserve* a book that's currently out.
- **What happens when a book isn't returned on time?** We need *fines*, calculated automatically based on how many days overdue a book is.
- **How do people find out about any of this?** *Notifications* — both inside the app, and by email, sent on a schedule.
- **What if the user's internet drops while they're borrowing a book?** *Offline support* — queue the action, replay it later.
- **How do we ship this to real users?** As a website, a Windows/Linux desktop app, and an Android/iOS app — from **one codebase**.
- **How do people in different countries use it?** *Internationalization* (i18n) — the same screens, in English or French (or any language you add).
- **How does it go from your laptop to a server the public can reach?** *Containerization* (Docker) and *CI/CD* (GitHub Actions).

Libroware solves all of these with a deliberately small, consistent set of tools. That consistency is the whole point of this course: once you understand *the pattern*, you can apply it to every feature, in every part of the app, forever.

0.2 The big picture



One frontend codebase (React + TypeScript, built with Vite). One backend codebase (Node.js + Apollo Server, talking GraphQL). One database (PostgreSQL, spoken to via Prisma, never with raw SQL). Everything else — Docker, Nginx, GitHub Actions, Electron, Capacitor — exists only to get that same triangle running in more places.

0.3 Tools to install before Part 2

Install these now. We'll explain what each one does in Part 1.

Tool	Why	Check it worked
Node.js (v20 LTS)	Runs both our backend and our frontend build tools	<code>node -v</code>
Git	Version control	<code>git --version</code>
VS Code (or any editor)	Where you'll write code	—
Docker Desktop	Runs PostgreSQL without installing it natively, and later packages the whole app	<code>docker --version</code>
A GraphQL client (optional) — Postman or Apollo Sandbox (the latter is built into Apollo Server and needs no install)	Manually testing the API before the frontend exists	—

0.4 The exercise convention

Every exercise box looks like this:

✂ **Exercise 0.1 — Sanity check** Open a terminal and run `node -v`, `git --version`, and `docker --version`. All three should print a version number. If `docker` fails, open Docker Desktop once to finish its setup, then retry.

Do the exercise *before* reading the next section. The course is sequential — each part's exercise produces the files the next part edits.

Part 1 — Foundations for True Beginners

If you already know what npm, REST, SQL migrations, and JWTs are, skim this part for the *Libroware-specific* framing and jump to Part 2.

1.1 How a web app actually talks

Every interaction in Libroware — logging in, borrowing a book, switching language — is your browser sending an HTTP **request** to a server, and the server sending back an HTTP **response**. That's true whether the API is REST or GraphQL; it's just *what's inside* the request that differs (we cover that in 1.5).

A request has: - A **method** (`GET`, `POST`, ...) - A **URL** (e.g. `http://localhost:5000/graphql`) - **Headers** (metadata — e.g. `Authorization: Bearer <token>`, or a cookie) - A **body** (the actual data, usually JSON)

A response has a **status code** (`200` OK, `401` Unauthorized, `500` Server Error, ...) and a **body**.

1.2 Node.js, npm, and `package.json`

Node.js is a runtime that lets you execute JavaScript outside a browser — on a server, or as a build tool on your own machine. Libroware's backend is a Node.js program; the frontend uses Node.js tools (Vite, TypeScript) to compile React/TypeScript into plain JS/HTML/CSS that a browser can run.

npm (Node Package Manager) ships with Node.js. It does two things: 1. Installs third-party code ("packages") that other people wrote, so you don't reinvent JWT signing or HTTP servers. 2. Runs scripts you define yourself.

Every Node project has a `package.json` — a manifest describing the project. Open `backend/package.json` in this repo and you'll see:

```
{
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "seed": "node prisma/seed.js"
  },
  "dependencies": {
    "@apollo/server": "^4.10.0",
    "@prisma/client": "^6.6.0",
    "bcryptjs": "^2.4.3"
  }
}
```

- `"dependencies"` lists packages your *running* program needs (downloaded into a `node_modules/` folder by `npm install`).
- `"scripts"` are named shell commands. `npm run dev` literally runs `nodemon src/index.js`. `npm start` runs `npm run start`, which npm lets you abbreviate without the word `run`.
- The `^` in `"^4.10.0"` means "any compatible version starting from 4.10.0" — npm resolves the exact version into `package-lock.json`, which you commit to Git so every developer (and every server) installs *identical* versions.

✂ **Exercise 1.1** Create an empty folder `playground/`. Run `npm init -y` inside it — this generates a minimal `package.json`. Open it and read every field. Then run `npm install left-pad` (a tiny, harmless real npm package) and look at what appeared: a `node_modules/` folder and a `package-lock.json`. Delete the folder when done — this was just to see the mechanics.

1.3 Git, in the amount you need

Git tracks changes to files over time as a sequence of **commits** — named snapshots. The commands you'll use constantly in this course:

```
git init                # start tracking a new project
git add <file>          # stage a change
git commit -m "message" # save a snapshot of staged changes
git status              # what's changed since the last commit
git log --oneline       # history of commits
```

You do not need branches, merges, or remotes to follow this course — just commit after each working exercise, so you can always go back.

✂ **Exercise 1.2** In a new folder, run `git init`, create a file `hello.txt` with any text, then `git add hello.txt && git commit -m "first commit"`. Run `git log` to see it.

1.4 Relational databases & why migrations exist

A **relational database** (PostgreSQL, in our case) stores data in **tables** — rows and columns, like a spreadsheet, but with the ability to *relate* rows in one table to rows in another (a `Borrow` row points at *which* `User` and *which* `Book` it's about).

The shape of those tables — which columns exist, their types, which tables relate to which — is the **schema**. Schemas change as an app grows (you add a `Fine` table; you add a `deletedAt` column to `User` for soft deletes). A **migration** is a recorded, ordered, repeatable script that takes a database from one schema version to the next. Instead of you manually running `ALTER TABLE` on every server by hand (and forgetting one, and breaking production), a migration tool generates and replays these scripts for you, in order, on every environment — your laptop, your teammate's laptop, and the production server — so they all end up with *exactly* the same schema. We use **Prisma Migrate** for this (Part 3).

1.5 REST vs GraphQL — why Libroware uses GraphQL

A traditional **REST** API exposes many URLs, one per resource: `GET /books`, `GET /books/:id`, `POST /books`, `GET /books/:id/reviews`, ... Each endpoint returns a fixed shape of data. If a screen needs a book *and* its author *and* its reviews, you either make three requests, or the backend team builds you a bespoke `GET /books/:id/full` endpoint.

GraphQL exposes exactly **one** URL (`/graphql`). Instead of picking a URL, the client sends a *query* describing exactly what data it wants, and the server returns exactly that shape — no more, no less:

```
query {
  book(id: "123") {
    title
    authors { name }
    reviews { rating comment }
  }
}
```

One request, one round trip, no over-fetching, no under-fetching. The tradeoff: GraphQL needs more upfront design (a **schema**, Part 4.1) and a slightly heavier server setup. For an app with many interrelated entities — books, authors, categories, borrows, fines, reservations — that upfront cost pays for itself fast, which is why Libroware is built this way.

1.6 JavaScript vs TypeScript

TypeScript is JavaScript with optional type annotations, checked *before* your code runs (at compile time), not while it's running. Plain JavaScript:

```
function addDays(date, days) {  
  return date + days; // bug: silently does string-ish math if date isn't a Date  
}
```

TypeScript:

```
function addDays(date: Date, days: number): Date {  
  const result = new Date(date);  
  result.setDate(result.getDate() + days);  
  return result;  
}
```

If you call `addDays("not a date", 3)`, TypeScript refuses to compile. That single property — catching mistakes before the app ever runs — is why both the Libroware frontend (Part 5) and, to a lesser extent, its tooling lean on TypeScript. The backend in this project is plain JavaScript (Node has no built-in TS support without a build step), which is a perfectly normal, common choice — TypeScript on the backend is an option, not a requirement.

1.7 Authentication vs authorization, and what a JWT is

- **Authentication**: proving *who* you are (logging in with an email + password).
- **Authorization**: deciding *what you're allowed to do* once we know who you are (only a librarian can delete a book).

A **JWT (JSON Web Token)** is a compact, signed string the server hands you after you log in successfully. It encodes claims (e.g. `{ userId, role }`) and is **signed** with a secret only the server knows — so the server can later verify the token wasn't tampered with, without needing to look anything up in a database. You send the JWT back on every subsequent request (as a cookie or a header), and the server decodes + verifies it to know who's asking. We build this exact flow in Part 4.3.

Part 2 — Project Setup & Monorepo Structure

2.1 Why one repository, two folders

Libroware keeps its backend and frontend in **one Git repository**, in two top-level folders:

```
libroware/
├── backend/    ← Node.js + Apollo Server + Prisma
├── frontend/   ← React + TypeScript + Vite
├── docker-compose.yml
└── .github/workflows/
```

This is called a **monorepo**. It's a choice, not a law — plenty of real systems split frontend and backend into separate repos. The monorepo wins here because both halves evolve together (a new GraphQL field is useless until the frontend queries it) and because it lets a single `docker-compose.yml` and a single CI pipeline describe the *whole* system in one place.

✂ **Exercise 2.1 — Scaffold your own monorepo** Create a folder `mylibrary/`. Inside it, run `git init`. Create two empty subfolders, `backend/` and `frontend/`. This is the skeleton you'll fill in for the rest of the course.

2.2 Initializing the backend

```
cd mylibrary/backend
npm init -y
```

Now install what the backend needs, exactly as Libroware does:

```
npm install express @apollo/server @apollo/server/express4 cors cookie-parser \
  jsonwebtoken bcryptjs dotenv express-rate-limit @prisma/client cloudinary \
  node-cron nodemailer
npm install --save-dev prisma nodemon
```

What each one is for (you'll meet all of them in Part 4):

Package	Role
<code>express</code>	The HTTP server framework that Apollo Server attaches to
<code>@apollo/server</code>	The GraphQL server itself
<code>cors</code>	Lets the frontend (a different origin) call the API

Package	Role
<code>cookie-parser</code>	Reads the <code>httpOnly</code> auth cookie off incoming requests
<code>jsonwebtoken</code>	Signs and verifies JWTs
<code>bcryptjs</code>	One-way hashes passwords (never store plain-text passwords)
<code>dotenv</code>	Loads <code>.env</code> files into <code>process.env</code>
<code>express-rate-limit</code>	Throttles abusive request bursts
<code>@prisma/client</code> + <code>prisma</code>	Talks to PostgreSQL; generates/runs migrations
<code>cloudinary</code>	Image hosting for profile pictures & book covers
<code>node-cron</code>	Schedules the daily notification job
<code>nodemailer</code>	Sends the reminder emails

Add the scripts from §1.2 to your new `package.json`'s `"scripts"` block.

2.3 Initializing the frontend

Libroware's frontend is built with **Vite** — a fast dev server and bundler — using the React + TypeScript template:

```
cd mylibrary/frontend
npm create vite@latest . -- --template react-ts
npm install
```

Then add the libraries the real Libroware frontend depends on:

```
npm install @apollo/client graphql react-router-dom react-i18next i18next \
  apollo3-cache-persist
```

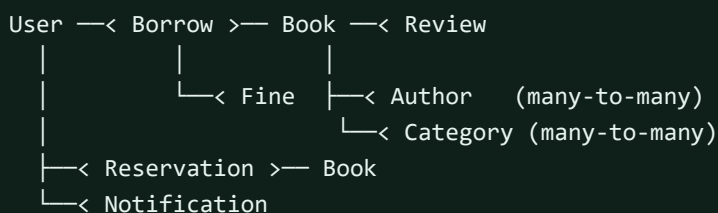
Package	Role
<code>@apollo/client</code> + <code>graphql</code>	The GraphQL client — sends queries/mutations, manages a local cache
<code>react-router-dom</code>	Client-side routing (<code>/profile</code> , <code>/admin</code> , ...) without full page reloads
<code>react-i18next</code> + <code>i18next</code>	Internationalization (Part 6)
<code>apollo3-cache-persist</code>	Persists the Apollo cache to <code>localStorage</code> so it survives a refresh

✂ **Exercise 2.2** Run `npm run dev` inside `mylibrary/frontend`. Confirm Vite's default React starter page loads in your browser at the printed `localhost` URL. This confirms your toolchain works before we write a single line of *our* code.

Part 3 — Designing the Database (Prisma + PostgreSQL)

3.1 Modeling the domain

Before writing any code, draw the *entities* and how they relate. Libroware's core domain:



- A **User** can have many **Borrow** records (one per book they've borrowed, ever).
- A **Borrow** belongs to exactly one **User** and one **Book**.
- A **Borrow** may produce exactly one **Fine**, if returned late.
- A **Book** can have many **Authors**, and an **Author** can have written many **Books** — a *many-to-many* relationship. Same for **Book** ↔ **Category**.
- A **User** can **Reservation**-queue for a **Book** that's currently unavailable.
- A **User** receives **Notifications** (in-app messages).

This diagram, drawn *before* you touch Prisma, is the most valuable five minutes of the whole project. Every bug in a real system traces back to a relationship someone didn't think through up front.

3.2 Running PostgreSQL locally with Docker

You don't need to install PostgreSQL natively — run it in a container:

```
docker run --name libroware-db -e POSTGRES_PASSWORD=devpass \
-e POSTGRES_DB=library_db -p 5432:5432 -d postgres:16-alpine
```

This pulls the official Postgres 16 image and starts it, exposing port 5432 on your machine.

`docker ps` should show it running.

3.3 Setting up Prisma

```
cd mylibrary/backend  
npx prisma init
```

This creates `prisma/schema.prisma` and a `.env` with a `DATABASE_URL` placeholder. Point it at the container you just started:

```
DATABASE_URL="postgresql://postgres:devpass@localhost:5432/library_db"
```

3.4 Writing the schema — the Author/Category/Book trio

Here is the **exact** structure Libroware uses for its simplest models. Open `backend/prisma/schema.prisma` in this repo side-by-side and follow along.

```

generator client {
  provider = "prisma-client-js"
  output   = "../generated/prisma"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Author {
  id          String   @id @default(uuid())
  name        String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  books       Book[]   @relation("BookToAuthor")
}

model Category {
  id          String   @id @default(uuid())
  name        String   @unique
  description  String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  books       Book[]   @relation("BookToCategory")
}

model Book {
  id          String   @id @default(uuid())
  title       String
  isbn        String   @unique
  description  String?
  publishedAt DateTime
  pageCount   Int
  quantity    Int       @default(1)
  available   Int       @default(1)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  authors     Author[]  @relation("BookToAuthor")
  categories  Category[] @relation("BookToCategory")
}

```

Line-by-line, the things that matter most for a beginner:

- `@id @default(uuid())` — the primary key. Libroware uses random UUID strings, not auto-incrementing integers, so IDs are unguessable and safe to generate on the client if ever needed.
- `String?` — the `?` means *nullable*: `description` is optional, `name` is not.
- `@unique` — the database enforces no two rows share this value (`isbn` , `Category.name`).
- `DateTime @default(now())` / `@updatedAt` — Prisma fills these automatically; you never set them by hand.

- `Book[] @relation("BookToAuthor")` — this is a **many-to-many** relation. Because *both* sides declare an array (`Author.books: Book[]` and `Book.authors: Author[]`), Prisma silently creates a hidden join table (`_BookToAuthor`) for you — no extra model needed. The matching `@relation("BookToAuthor")` name on both sides is what tells Prisma these two array fields describe the *same* relationship (useful once you have more than one relation between the same two models).

✂ **Exercise 3.1** Add the three models above to your own `schema.prisma`. Then add a `User` model with `id`, `email (@unique)`, `password`, `firstName`, `lastName`, and a `role` field using an **enum**: `prisma enum Role { USER LIBRARIAN ADMIN }` Give `User` a `role Role @default(USER)` field.

3.5 Running your first migration

```
npx prisma migrate dev --name init
```

This does three things: (1) generates a SQL migration file under `prisma/migrations/<timestamp>_init/`, (2) applies it to your database, (3) regenerates the **Prisma Client** — a fully-typed JavaScript/TypeScript API for querying your exact schema, written into `generated/prisma` (per the `output` path we set in `generator client`).

Open the generated `.sql` migration file. Read it. This is the literal `CREATE TABLE` statement Prisma derived from your schema — there is no magic here, just a tool saving you from writing this by hand and remembering to run it everywhere.

✂ **Exercise 3.2** Run the migration. Then open `npx prisma studio` — a browser-based GUI Prisma ships for free — and manually add one `Author` row through the UI. Confirm it shows up if you query it (next chapter will do this from code).

3.6 The other models, briefly

The full Libroware schema also has `Borrow`, `Fine`, `Reservation`, `Review`, `Notification`, and `AuditLog`. They follow the *exact same patterns* you just learned — foreign keys via `fields: [...]` references: `[...]`, enums for fixed states, indexes (`@@index([...])`) on columns you'll filter by often (e.g. `@@index([userId])` on `Borrow`, so "find all borrows for this user" stays fast as the table grows). We will add `Borrow` and `Fine` ourselves in the Part 4 exercises, once you've seen the resolver pattern that uses them.

Part 4 — Building the GraphQL API

This is the heart of the backend. We build it in the same order Libroware itself is structured: schema first, then the Express/Apollo wiring, then auth, then a full CRUD resolver, walked line by line.

4.1 Schema-first GraphQL

In **schema-first** GraphQL (what Libroware uses), you write the *shape* of your API as a `.graphql` file *before* writing any resolver logic. The file declares **types** (the shape of data), **inputs** (the shape of arguments), and two special root types: **Query** (read operations) and **Mutation** (write operations).

Here is the real `Author` -related slice of `backend/src/graphql/schema.graphql` :

```
type Author {
  id: ID!
  name: String!
  books: [Book!]!
  createdAt: String!
  updatedAt: String!
}

input AuthorCreateInput {
  name: String!
}

input AuthorUpdateInput {
  name: String
}

type Query {
  author(id: ID!): Author
  authors(skip: Int, take: Int, searchName: String): [Author!]!
}

type Mutation {
  createAuthor(input: AuthorCreateInput!): Author!
  updateAuthor(id: ID!, input: AuthorUpdateInput!): Author!
  deleteAuthor(id: ID!): Author!
}
```

Notes on the syntax: - `String!` — the `!` means *non-nullable*: this field is guaranteed to never be `null`. Without `!`, it's optional. - `[Book!]!` — a non-nullable array of non-nullable `Book`s. Read array types inside-out: "an array of `Book` (each guaranteed non-null), and the array itself is guaranteed non-null." - **Inputs are separate from types.** `AuthorCreateInput` requires `name`; `AuthorUpdateInput` makes it optional (you might only be updating *some* fields). This

separation — output shape vs. input shape — is a GraphQL convention worth internalizing early.

- `Query` and `Mutation` are not special syntactically — they're just types — but Apollo Server treats them as the two entry points into your API.

✂ **Exercise 4.1** Create `backend/src/graphql/schema.graphql` in your project. Copy the `Author` block above. Add the matching block for `Category` (it has `name`, `description`, `books`) yourself, following the same shape.

4.2 Wiring Express + Apollo Server

Here is the real `backend/src/index.js`, trimmed to its essential skeleton (the full file also adds rate limiting and audit logging, covered in §4.2.1 and Part 8.4):

```

const express = require("express");
const { ApolloServer } = require("@apollo/server");
const { expressMiddleware } = require("@apollo/server/express4");
const { readFileSync } = require("fs");
const { join } = require("path");
const cors = require("cors");
const cookieParser = require("cookie-parser");
const jwt = require("jsonwebtoken");
const resolvers = require("../graphql/resolvers");
const { PrismaClient } = require("../generated/prisma");

require("dotenv").config();

const typeDefs = readFileSync(join(__dirname, "graphql", "schema.graphql"), "utf8");

const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) throw new Error("JWT_SECRET environment variable is required");

const getUser = (token) => {
  if (!token) return { userId: null, role: null };
  try {
    const { userId, role } = jwt.verify(token, JWT_SECRET);
    return { userId, role };
  } catch (error) {
    return { userId: null, role: null }; // expired/invalid token → treat as logged out
  }
};

const prisma = new PrismaClient();

async function startServer() {
  const app = express();
  app.use(cookieParser());

  const server = new ApolloServer({ typeDefs, resolvers });
  await server.start();

  app.use(
    "/graphql",
    cors({ origin: process.env.ALLOWED_ORIGINS.split(","), credentials: true }),
    express.json(),
    expressMiddleware(server, {
      context: async ({ req, res }) => {
        const cookieToken = req.cookies?.auth_token;
        const bearerToken = req.headers.authorization?.replace("Bearer ", "") || null;
        const { userId, role } = getUser(cookieToken || bearerToken);
        return { userId, role, prisma, res };
      },
    })
  );

  app.listen(process.env.PORT || 4000);
}

```

```
startServer();
```

Walking through *why* each piece exists:

- `readFileSync(...schema.graphql)` — the schema you wrote in 4.1 is loaded as plain text and handed to Apollo, which parses it into an internal representation.
- `getUser(token)` — this is the entire authentication check for *every single request*. It tries to verify the JWT; if it's missing, expired, or tampered with, `jwt.verify` throws, we catch it, and we return `{ userId: null, role: null }` — i.e., "treat this request as anonymous." Note this function never throws or rejects the request itself — it just figures out *who's asking*. Whether that's *enough* to proceed is the resolver's job (§4.4).
- **The context function** — this is the bridge between Express (the HTTP layer) and your resolvers (the GraphQL layer). Apollo Server calls it once per request, and *whatever object it returns* is the third argument every resolver receives. This is how `prisma`, `userId`, `role`, and `res` (the Express response object, needed to set cookies) become available everywhere, without you passing them around manually.
- `cookieParser()` and reading `req.cookies?.auth_token` — Libroware supports two ways of sending the JWT: an **httpOnly cookie** (used by the web app — the browser attaches it automatically and JavaScript can never read it, which blocks an entire class of token-theft attacks) and a **Bearer header** (used by the Electron/Capacitor builds, where there's no browser cookie jar in the same way). The code prefers the cookie, falling back to the header.
- `cors({ origin: ..., credentials: true })` — by default, a browser blocks JavaScript on `http://localhost:3000` from calling `http://localhost:4000` (a different *origin*). CORS is the server explicitly opting specific origins back in. `credentials: true` is required for cookies to be sent cross-origin at all.

✂ **Exercise 4.2** Write your own `backend/src/index.js` using the skeleton above. Create a `backend/src/graphql/resolvers/index.js` that, for now, just exports an empty `{ Query: {}, Mutation: {} }`. Add a `.env` with `JWT_SECRET=anything-for-now`, `DATABASE_URL=...` (from Part 3), and `ALLOWED_ORIGINS=http://localhost:5173` (Vite's default port). Run `npm run dev` and confirm the server starts without crashing.

4.3 Authentication: signup, login, logout

Now the resolvers that issue and clear that JWT. This is the **exact** logic from `backend/src/graphql/resolvers/user.js` :

```

const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const JWT_SECRET = process.env.JWT_SECRET;

const COOKIE_OPTS = {
  httpOnly: true,
  sameSite: "strict",
  secure: process.env.COOKIE_SECURE === "true", // only over HTTPS once you have TLS
  maxAge: 7 * 24 * 60 * 60 * 1000,           // 7 days – matches JWT expiry
  path: "/",
};

function setAuthCookie(res, token) {
  if (res) res.cookie("auth_token", token, COOKIE_OPTS);
}

module.exports = {
  Mutation: {
    signup: async (_, { input }, { prisma, res }) => {
      const { email, password, firstName, lastName } = input;

      if (!password || password.length < 8)
        throw new Error("Password must be at least 8 characters");

      const existing = await prisma.user.findUnique({ where: { email } });
      if (existing) throw new Error("User already exists with this email");

      const hashedPassword = await bcrypt.hash(password, 10);
      const user = await prisma.user.create({
        data: { email, password: hashedPassword, firstName, lastName },
      });

      const token = jwt.sign(
        { userId: user.id, email: user.email, role: user.role },
        JWT_SECRET,
        { expiresIn: "7d" }
      );

      setAuthCookie(res, token);
      return { token, user };
    },

    login: async (_, { input }, { prisma, res }) => {
      const { email, password } = input;
      const user = await prisma.user.findUnique({ where: { email } });
      if (!user) throw new Error("No user found with this email");

      const validPassword = await bcrypt.compare(password, user.password);
      if (!validPassword) throw new Error("Invalid password");

      const token = jwt.sign(
        { userId: user.id, email: user.email, role: user.role },
        JWT_SECRET,
        { expiresIn: "7d" }
      );
    },
  },
};

```

```

    );

    setAuthCookie(res, token);
    return { token, user };
  },

  logout: (_, __, { res }) => {
    if (res) res.clearCookie("auth_token", { path: "/" });
    return true;
  },
},
};

```

The two ideas that matter most here:

1. **We never store the password.** `bcrypt.hash(password, 10)` runs a one-way, deliberately slow hashing algorithm — `10` is the "cost factor," how many times it iterates internally (higher = slower = harder to brute-force, at the cost of slower logins). We store only the hash. On login, `bcrypt.compare(password, user.password)` re-hashes the attempt and compares — it never "decrypts" anything, because hashing isn't reversible. This is the only correct way to store passwords; never roll your own.
2. **The JWT and the cookie are two different things.** `jwt.sign(...)` produces the token string itself. `setAuthCookie` is what *delivers* it to the browser, as an `httpOnly` cookie the browser will automatically attach to every future request to this domain. The mutation *also* returns the raw token in `{ token, user }` — that's for the Electron/Capacitor clients, which store it themselves (Part 9) and send it back as a `Bearer` header instead of relying on cookies.

Add the matching schema:

```

input LoginInput { email: String!, password: String! }
type AuthPayload { token: String!, user: User! }

type Mutation {
  signup(input: UserCreateInput!): AuthPayload!
  login(input: LoginInput!): AuthPayload!
  logout: Boolean!
}

```

✂ **Exercise 4.3** Add `signup`, `login`, `logout` to your schema and resolvers. Start your server, open `http://localhost:4000/graphql` in a browser (Apollo Server serves a built-in sandbox UI there in development), and run: `graphql mutation { signup(input: { email: "you@example.com", password: "password123", firstName: "A", lastName: "B" }) { token user { id email } } }` Confirm you get back a token and a user. Then run the equivalent `login` mutation with the same credentials.

4.4 Authorization: the pattern repeated everywhere

Every resolver in Libroware that requires being logged in, or having a specific role, starts with the same two-line guard. From `backend/src/graphql/resolvers/author.js` :

```
createAuthor: async (_, { input }, { userId, role, prisma }) => {
  if (!userId) throw new Error("Not authenticated");
  if (role !== "ADMIN" && role !== "LIBRARIAN")
    throw new Error("Only librarians and admins can create authors");

  const { name } = input;
  if (!name || !name.trim()) throw new Error("Author name is required");

  return prisma.author.create({ data: { name: name.trim() } });
},
```

This is the entire authorization model of the app: no middleware magic, no decorators, no separate permissions library — just plain `if` statements at the top of each resolver, checking the `userId` and `role` that the `context` function (§4.2) already resolved from the JWT. It's simple enough that anyone reading the resolver can see *exactly* who's allowed to call it, without chasing definitions across five files. That legibility is worth more than a clever abstraction, especially for a small team.

Note throwing a plain `Error("Not authenticated")` (rather than a special "401" error) is a deliberate, simple choice in this codebase — the frontend (Part 7) recognizes these specific messages to decide when to force a logout, instead of relying on an HTTP status code.

4.5 A full CRUD resolver, line by line

This is the complete, real `backend/src/graphql/resolvers/author.js` — the simplest full CRUD module in the project, and the one we'll mirror by hand in the exercise.

```

module.exports = {
  Query: {
    author: async (_, { id }, { prisma }) => {
      return prisma.author.findUnique({ where: { id } });
    },

    authors: async (_, { skip = 0, take = 10, searchName }, { prisma }) => {
      const where = searchName
        ? { name: { contains: searchName, mode: "insensitive" } }
        : {};

      return prisma.author.findMany({
        where,
        skip,
        take,
        orderBy: { name: "asc" },
      });
    },
  },

  Mutation: {
    createAuthor: async (_, { input }, { userId, role, prisma }) => {
      if (!userId) throw new Error("Not authenticated");
      if (role !== "ADMIN" && role !== "LIBRARIAN")
        throw new Error("Only librarians and admins can create authors");

      const { name } = input;
      if (!name || !name.trim()) throw new Error("Author name is required");

      return prisma.author.create({ data: { name: name.trim() } });
    },

    updateAuthor: async (_, { id, input }, { userId, role, prisma }) => {
      if (!userId) throw new Error("Not authenticated");
      if (role !== "ADMIN" && role !== "LIBRARIAN")
        throw new Error("Only librarians and admins can update authors");

      const author = await prisma.author.findUnique({ where: { id } });
      if (!author) throw new Error("Author not found");

      return prisma.author.update({ where: { id }, data: input });
    },

    deleteAuthor: async (_, { id }, { userId, role, prisma }) => {
      if (!userId) throw new Error("Not authenticated");
      if (role !== "ADMIN" && role !== "LIBRARIAN")
        throw new Error("Only librarians and admins can delete authors");

      const author = await prisma.author.findUnique({ where: { id } });
      if (!author) throw new Error("Author not found");

      const authorBooks = await prisma.book.findMany({
        where: { authors: { some: { id } } },
      });
    },
  },
};

```

```

    if (authorBooks.length > 0)
      throw new Error("Cannot delete author because they have associated books");

    return prisma.author.delete({ where: { id } });
  },
},

// A resolver for a *field*, not a root Query/Mutation – see explanation below.
Author: {
  books: async (parent, _, { prisma }) => {
    return prisma.book.findMany({
      where: { authors: { some: { id: parent.id } } },
      include: { authors: true, categories: true },
    });
  },
},
};

```

Three things to understand deeply here, because they generalize to *every other resolver in the app*:

1. Every resolver receives the same four arguments: `(parent, args, context, info)`.

Most of ours ignore `parent` (written `_`) and `info` entirely. `args` is whatever the client passed in the GraphQL query/mutation (`{ id }`, `{ input }`, `{ skip, take, searchName }`).

`context` is that object we built in §4.2 (`{ userId, role, prisma, res }`).

2. The `Author: { books: ... }` block is a *field resolver*, not a root resolver. When a client asks for `author(id: "x") { name books { title } }`, Apollo first runs the `Query.author` resolver to get the base `Author` row, *then*, because the client also asked for `books`, Apollo calls `Author.books(parent, ...)` — where `parent` is that `Author` row Prisma just returned. This is GraphQL's core trick: every field in a query can have its own resolver, and they compose automatically. You only write a field resolver when the data isn't already sitting on the parent object (here, the raw `Author` row has no `books` array — Prisma doesn't include relations unless you ask — so we fetch them explicitly).

3. Defensive checks before destructive operations. `deleteAuthor` doesn't just call `prisma.author.delete(...)` — it first checks whether any `Book` still references this author, and refuses if so. This is a *business rule* ("you can't orphan a book's author"), and it lives in the resolver, not the database — Prisma's relation alone wouldn't stop this delete (there's no foreign key constraint *from* `Book` to `Author` in a many-to-many relation the way there is in a one-to-many).

✂ Exercise 4.4 — Build CategoryManagement 's backend twin

Write `backend/src/graphql/resolvers/category.js` from scratch, mirroring `author.js` exactly, but for `Category` (which additionally has a `description` field). Wire it into `backend/src/graphql/resolvers/index.js` by merging its `Query` / `Mutation` objects with the others (Libroware's real `resolvers/index.js` does this with a small merge helper — open it in this repo to see the pattern). Test all four operations (`category` , `categories` , `createCategory` , `updateCategory` , `deleteCategory`) in the Apollo sandbox before moving to Part 5.

Part 5 — Building the Frontend

We now build the React side that talks to the API from Part 4 — starting with the plumbing (Apollo Client, routing, auth context), then a full CRUD screen end-to-end.

5.1 The entry point

`frontend/src/main.tsx` is where the React app boots:

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { ApolloProvider } from "@apollo/client";
import { client, initCache } from "@apollo-client";
import App from "@App";
import "@i18n";

initCache().then(() => {
  ReactDOM.createRoot(document.getElementById("root")!).render(
    <React.StrictMode>
      <ApolloProvider client={client}>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </ApolloProvider>
    </React.StrictMode>
  );
});
```

Read this from the inside out: `<App />` is your whole UI. `<BrowserRouter>` gives every component inside it access to client-side routing (`useNavigate` , `<Link>` , ...). `<ApolloProvider client={client}>` gives every component access to `useQuery` / `useMutation` against the GraphQL API. `initCache().then(...)` delays the very first render until the persisted Apollo

cache (Part 8.1) has loaded from disk — so a returning user sees their last-known data instantly, instead of a blank loading flash.

5.2 Apollo Client: the link chain

`frontend/src/apollo-client.ts` configures *how* every GraphQL request leaves the browser. The key concept is a **link chain** — middleware functions composed together, each request passing through all of them in order:

```
import { ApolloClient, InMemoryCache, from } from '@apollo/client';
import { HttpLink } from '@apollo/client/link/http';
import { setContext } from '@apollo/client/link/context';
import { onError } from '@apollo/client/link/error';

const errorLink = onError(({ graphQLErrors, networkError }) => {
  if (graphQLErrors) {
    graphQLErrors.forEach(({ message }) => console.error('[GraphQL error]:', message));
  }
  if (networkError) console.error('[Network error]:', networkError);
});

const httpLink = new HttpLink({
  uri: '/api/graphql', // proxied to the backend — see Part 10.2
  credentials: 'include', // send the httpOnly cookie on every request
});

const authLink = setContext( (_, { headers }) => {
  const token = localStorage.getItem('token'); // only set on Electron/Capacitor builds
  return {
    headers: {
      ...headers,
      Authorization: token ? `Bearer ${token}` : "",
    },
  };
});

export const client = new ApolloClient({
  link: from([errorLink, authLink, httpLink]),
  cache: new InMemoryCache(),
});
```

`from([errorLink, authLink, httpLink])` chains them: every outgoing request passes through `errorLink`, then `authLink` (which attaches the `Authorization` header, if any), then finally `httpLink`, which actually performs the network call. Errors flow back through the same chain in reverse. **Order matters** — `errorLink` has to wrap the others to see errors coming from *any* of them, and `httpLink` must be last because it's the one that actually talks to the network.

5.3 Auth context: the single source of truth for "am I logged in?"

`frontend/src/context/AuthContext.tsx` is a React Context — a way to make some piece of state available to *any* component in the tree, without manually passing it down as props through every layer in between.

```

import React, { createContext, useState, useContext, useEffect, ReactNode } from "react";

interface AuthContextType {
  isAuthenticated: boolean;
  user: User | null;
  login: (token: string, user: User) => void;
  logout: () => void;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: ReactNode }> = ({ children }) => {
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [user, setUser] = useState<User | null>(null);

  useEffect(() => {
    // Rehydrate session from localStorage on app start
    const storedUser = localStorage.getItem("user");
    if (storedUser) {
      setUser(JSON.parse(storedUser));
      setIsAuthenticated(true);
    }
  }, []);

  const login = (token: string, newUser: User) => {
    localStorage.setItem("user", JSON.stringify(newUser));
    setUser(newUser);
    setIsAuthenticated(true);
  };

  const logout = () => {
    localStorage.removeItem("user");
    setUser(null);
    setIsAuthenticated(false);
  };

  return (
    <AuthContext.Provider value={{ isAuthenticated, user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = (): AuthContextType => {
  const context = useContext(AuthContext);
  if (context === undefined) throw new Error("useAuth must be used within an AuthProvider");
  return context;
};

```

Every component that needs to know "is someone logged in, and who" calls `const { isAuthenticated, user } = useAuth();` — no prop drilling, no passing `user` through ten layers of components that don't otherwise need it.

`App.tsx` uses exactly this value to decide what to render — this is the entire "route guarding" mechanism in Libroware, no special library needed:

```
const AppContent: React.FC = () => {
  const { isAuthenticated } = useAuth();
  return isAuthenticated ? <AuthenticatedRoutes /> : <LoginPage />;
};
```

If you're not authenticated, you *only* ever see `<LoginPage />`, no matter what URL you typed. There is no separate "protected route" component to configure per-page — the gate sits once, at the top.

✂ **Exercise 5.1** Build `AuthContext.tsx` as above in your project. Build a minimal `LoginPage.tsx` with an email/password form that calls a `LOGIN` mutation (using Apollo's `useMutation` hook) against the `login` resolver from Part 4.3, and on success calls `login(data.login.token, data.login.user)` from `useAuth()`. Wrap your `<App />` in `<AuthProvider>` in `main.tsx`. Confirm: logging in shows a different screen than being logged out, and refreshing the page keeps you logged in (thanks to the `useEffect` rehydration).

5.4 A full CRUD screen, end to end

Now we connect everything: a React component that lists, creates, edits, and deletes `Author` rows through the API from Part 4.5. This is (slightly trimmed) the real `frontend/src/components/admin/AuthorManagement.tsx`.

Step 1 — define the GraphQL operations the component needs, typically in a shared `graphql/queries.ts` / `graphql/mutations.ts`:

```
import { gql } from '@apollo/client';

export const GET_AUTHORS = gql`
  query GetAuthors($skip: Int, $take: Int, $searchName: String) {
    authors(skip: $skip, take: $take, searchName: $searchName) {
      id
      name
      createdAt
    }
  }
`;

export const CREATE_AUTHOR = gql`
  mutation CreateAuthor($input: AuthorCreateInput!) {
    createAuthor(input: $input) { id name }
  }
`;

export const DELETE_AUTHOR = gql`
  mutation DeleteAuthor($id: ID!) {
    deleteAuthor(id: $id) { id }
  }
`;
```

Notice the query and the mutations only ask for the fields the screen actually needs (`id` , `name` , `createdAt`) — this is GraphQL's "ask for exactly what you want" promise in practice.

Step 2 — fetch and render the list, with `useQuery` :

```
const { loading, error, data, refetch } = useQuery(GET_AUTHORS, {
  variables: { skip: 0, take: 50, searchName: searchTerm || undefined },
});

if (loading) return <p>Loading authors...</p>;
if (error) return <p>Error loading authors: {error.message}</p>;

return (
  <ul>
    {data.authors.map((author) => (
      <li key={author.id}>{author.name}</li>
    ))}
  </ul>
);
```

`useQuery` automatically: fires the request when the component mounts, re-fires it if `variables` change (e.g. when `searchTerm` changes — this is *why* the search box "just works" as you type), tracks `loading` / `error` / `data` as React state, and caches the result so navigating away and back doesn't always re-fetch from scratch.

Step 3 — mutate, with `useMutation` , and refresh the list on success:

```
const [createAuthor, { loading: createLoading }] = useMutation(CREATE_AUTHOR, {
  onCompleted: () => {
    setIsFormModalOpen(false);
    refetch(); // re-run GET_AUTHORS to show the new row immediately
  },
  onError: (error) => setError(error.message),
});

const handleSubmit = () => {
  if (!formData.name.trim()) {
    setError("Author name is required");
    return;
  }
  createAuthor({ variables: { input: { name: formData.name } } });
};
```

`useMutation` returns a *tuple*: a function you call to fire the mutation (`createAuthor({ variables: ... })`), and a status object (`createLoading` , etc.) you use to disable the submit button while in flight. `onCompleted` / `onError` are callbacks Apollo invokes once the network round-trip settles — this is where you close the modal and refresh data, or surface a validation error from the server.

Step 4 — delete, with a confirmation step. Libroware never deletes on a single click — it opens a confirmation modal first:

```
const [deleteAuthor] = useMutation(DELETE_AUTHOR, { onCompleted: () => refetch() });
const [authorToDelete, setAuthorToDelete] = useState<string | null>(null);

// ... user clicks a trash icon → setAuthorToDelete(author.id) opens the modal ...

const confirmDelete = () => {
  if (authorToDelete) deleteAuthor({ variables: { id: authorToDelete } });
  setAuthorToDelete(null);
};
```

This separation — "intent to delete" (`authorToDelete` set) vs. "confirmed delete" (`confirmDelete()` called) — is what a `<DeleteConfirmation>` modal component renders against; it's a UI pattern, not a GraphQL one, but it's the pattern Libroware repeats for every destructive action in the app (deleting a book, a category, a user...).

✂ **Exercise 5.2 — Build `CategoryManagement.tsx`** Using the four steps above and the `Category` mutations/resolvers you wrote in Exercise 4.4, build a full list/create/edit/delete screen for categories. Reuse a generic `<Modal>` component for the create/edit form and a `<DeleteConfirmation>` component for deletes (or build minimal versions of your own — the real ones in `frontend/src/components/Modal.tsx` and `DeleteConfirmation.tsx` are good references once you've tried it yourself). Add a tab for it in whatever top-level admin screen you're building. By the end of this exercise you will have rebuilt, end-to-end, the *exact* feature pattern that every CRUD screen in Libroware uses — books, fines, reservations, reviews are all variations on this same shape.

Part 6 — Internationalization (i18n)

6.1 Why i18n is a day-one decision, not a day-1000 retrofit

Libroware ships in English and French. The mechanism is `react-i18next` : every piece of user-facing text is looked up by a *key* (`"profile.edit"`) rather than written inline (`"Edit Profile"`), and a JSON file per language maps keys to that language's text. Retrofitting this onto an app that was written with hardcoded strings everywhere is a slow, error-prone hunt — which is exactly the bug class we'll deliberately reproduce and fix in Part 7's spirit-sibling chapter here.

6.2 Setup

`frontend/src/i18n.ts` , the real file in this repo:

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import en from './locales/en.json';
import fr from './locales/fr.json';

const saved = localStorage.getItem('libroware_lang') || 'en';

i18n.use(initReactI18next).init({
  resources: {
    en: { translation: en },
    fr: { translation: fr },
  },
  lng: saved,
  fallbackLng: 'en',
  interpolation: { escapeValue: false },
});

export default i18n;
```

`resources` is just two plain JSON objects, imported like any other module. `fallbackLng: 'en'` means: if a key is missing in the active language, fall back to English rather than rendering a blank string. `escapeValue: false` disables i18next's own HTML-escaping of interpolated values — safe here because React already escapes everything it renders, so double-escaping isn't needed. This module is imported once, in `main.tsx` (§5.1, `import "@/i18n"`), purely for its side effect of calling `i18n.init(...)` — nothing is exported and used directly elsewhere.

6.3 Locale files and the `useTranslation` hook

`frontend/src/locales/en.json` is a nested JSON object, namespaced by feature:

```
{
  "nav": {
    "dashboard": "Dashboard",
    "yourProfile": "Your Profile",
    "signOut": "Sign out"
  },
  "authors": {
    "title": "Authors",
    "addNew": "Add Author",
    "nameRequired": "Author name is required"
  }
}
```

`frontend/src/locales/fr.json` mirrors the *exact same key structure*, with French values:

```
{
  "nav": {
    "dashboard": "Tableau de bord",
    "yourProfile": "Votre profil",
    "signOut": "Se déconnecter"
  }
}
```

In a component:

```
import { useTranslation } from "react-i18next";

const Navigation: React.FC = () => {
  const { t, i18n } = useTranslation();

  const toggleLanguage = () => {
    const next = i18n.language === 'en' ? 'fr' : 'en';
    i18n.changeLanguage(next);
    localStorage.setItem('libroware_lang', next);
  };

  return (
    <nav>
      <Link to="/profile">{t('nav.yourProfile')}</Link>
      <button onClick={toggleLanguage}>{t('nav.dashboard')}</button>
    </nav>
  );
};
```

`t('nav.yourProfile')` looks up that dotted path in the *currently active* language's JSON. `i18n.changeLanguage('fr')` swaps the active language **instantly, for the whole app** — every component using `t(...)` re-renders with the new strings, with no page reload, because `react-i18next` subscribes components to language changes under the hood.

For text with dynamic values, `i18next` supports interpolation directly in the JSON:

```
{ "profile": { "userNotFound": "User not found (ID: {{userId}})" } }
```

```
t('profile.userNotFound', { userId: "abc-123" })
// → "User not found (ID: abc-123)"
```

6.4 The bug we just fixed in this very codebase

While preparing this course, an audit of the real Libroware frontend found several components where a developer wrote plain JSX text instead of a `t(...)` call — for example, `Navigation.tsx` had:

```
// ❌ Before – hardcoded, ignores the active language entirely
<Link to="/profile">Your Profile</Link>
<button onClick={() => logout()}>Sign out</button>
```

instead of:

```
// ✅ After – respects whatever language the user picked
<Link to="/profile">{t('nav.yourProfile')}</Link>
<button onClick={() => logout()}>{t('nav.signOut')}</button>
```

The fix is mechanical once you know what to look for: **grep your component tree for quoted strings inside JSX** ("Your Profile" , "Sign out" , "Loading..." , placeholder="Search authors...") and ask, for each one, "does this need to change if the user switches language?" If yes, it needs a key in both locale files and a `t(...)` call. This exact sweep was performed across `Navigation.tsx` , `UserProfile.tsx` , `ProfileEditor.tsx` , and the admin management screens in this project — the same sweep you should run on your own rebuild.

✂ **Exercise 6.1** In your `CategoryManagement.tsx` from Exercise 5.2, find every hardcoded string (button labels, the search placeholder, the "no categories found" message, validation errors) and replace them with `t('categories.xxx')` calls, adding matching keys to both an `en.json` and a `fr.json` . Add a language-toggle button anywhere in your UI and confirm every string you touched changes when you switch languages — and that nothing you missed stays stuck in English.

Part 7 — Auth Hardening: Session Expiry

This chapter documents a real fix made to this exact codebase, because it's the single best illustration in the whole project of *why* you sometimes need to bridge non-React code and React state.

7.1 The problem

A JWT expires after 7 days (§4.3). Before this fix, here's what happened in Libroware when a request failed because the token had expired: the Apollo `errorLink` (§5.2) would `console.error` the failure... and nothing else. The UI didn't redirect anywhere. `AuthContext` 's `isAuthenticated` stayed `true` . The user sat on a broken screen, with every subsequent action silently failing, with no indication *why*, until they manually logged out and back in.

7.2 Why this is harder than it sounds

The fix obviously needs to call `logout()` from `AuthContext`. But `apollo-client.ts` — where the error is detected — is a **plain TypeScript module**, not a React component. It has no access to `useAuth()` (hooks only work inside components) and is, deliberately, initialized *before* any React component exists (§5.1 — `client` is created at module load time, then handed to `<ApolloProvider client={client}>`). There is no clean, direct function call from "inside Apollo's link chain" to "inside a React context provider."

7.3 The fix: a DOM event as the bridge

The browser's own event system doesn't care whether the code on either end is "in React" or not — `window` is global, and both plain modules and React components can listen on it and dispatch to it. That's the bridge:

In `apollo-client.ts` (plain module, detects the failure):

```
export const AUTH_EXPIRED_EVENT = 'libroware:auth-expired';

const isAuthError = (message: string) =>
  /not authenticated|not authorized|jwt expired|invalid token/i.test(message);

const errorLink = onError(({ graphqlErrors, networkError }) => {
  let authExpired = false;

  if (graphqlErrors) {
    graphqlErrors.forEach(({ message }) => {
      if (isAuthError(message)) authExpired = true;
    });
  }

  if (networkError) {
    const statusCode = (networkError as ServerError).statusCode;
    if (statusCode === 401 || statusCode === 403) authExpired = true;
  }

  if (authExpired) {
    window.dispatchEvent(new Event(AUTH_EXPIRED_EVENT));
  }
});
```

Notice `isAuthError` matches on the literal error *messages* the backend resolvers throw (`"Not authenticated"`, `"Not authorized"` — recall §4.4's plain `throw new Error(...)` pattern), not just HTTP status codes — because this GraphQL API returns its own errors inside a `200 OK` response body, not as distinct HTTP status codes, for most auth failures.

In `AuthContext.tsx` (a React component, reacts to it):

```
import { client, AUTH_EXPIRED_EVENT } from "../apollo-client";

export const AuthProvider: React.FC<{ children: ReactNode }> = ({ children }) => {
  // ...existing login/logout/state from Part 5.3...

  useEffect(() => {
    const handleAuthExpired = () => logout();
    window.addEventListener(AUTH_EXPIRED_EVENT, handleAuthExpired);
    return () => window.removeEventListener(AUTH_EXPIRED_EVENT, handleAuthExpired);
  }, []);

  // ...
};
```

And because `AppContent` (§5.3) already renders `<LoginPage />` whenever `isAuthenticated` is `false` — with *no other code changes needed* — calling `logout()` here automatically bounces the user back to the login screen the instant any request reveals their session is dead. The fix required exactly two small additions, because the rest of the architecture (the auth gate built in Part 5) was already shaped to make this trivial once the signal existed.

7.4 The general lesson

This is the standard escape hatch whenever you need to notify React state from outside the component tree — a WebSocket message handler, a Service Worker, a third-party SDK callback, a browser API event. `window.dispatchEvent(new Event("my-event"))` / `window.addEventListener("my-event", handler)` costs nothing, requires no library, and works regardless of which framework (or no framework) is on either side.

✂ **Exercise 7.1** In your own rebuild, deliberately wait for your JWT to expire (or temporarily hardcode a 10-second `expiresIn` in `jwt.sign` to test faster), then try to fetch the author list. Confirm you currently see only a silent console error. Implement the `AUTH_EXPIRED_EVENT` pattern above. Confirm that, this time, you're automatically returned to your login screen the moment the expired token is used.

Part 8 — Cross-Cutting Concerns (Survey)

These are real, working subsystems in Libroware. We cover them at *survey* depth — enough to understand the architecture and read the real source confidently — rather than rebuilding them by hand.

8.1 Offline support

Problem: a user borrowing a book on a spotty connection shouldn't lose the action.

Approach: `frontend/src/offline/offlineQueue.ts` keeps a small FIFO queue in `localStorage`. `useOfflineMutation.ts` wraps Apollo's `useMutation`: if the app is online, it just calls the mutation normally; if offline, it applies an *optimistic* update directly to the Apollo cache (so the UI looks correct immediately) and pushes the mutation's variables onto the queue instead of sending them:

```
const run = (args: { variables: TVariables }) => {
  if (isOnline) { mutate(args); return; }
  config.applyOptimistic(client.cache, args.variables);
  enqueue({ type: config.type, variables: args.variables });
  addToast(config.queuedMessage, "info");
};
```

When `NetworkContext.tsx` detects the browser came back online, `replayQueue.ts` walks the queue and calls `client.mutate(...)` for each entry in order, stopping on the first failure to avoid replaying out of sequence, then triggers `client.reFetchObservableQueries()` to reconcile the UI with whatever the server's real state turned out to be.

Separately, `apollo3-cache-persist` (already wired up in §5.2's `client`) writes the entire Apollo cache to `localStorage` after every change and reloads it on startup (`initCache()` in §5.1) — so closing the tab and reopening it shows your last-seen data instantly, even before the network request resolves.

8.2 Notifications: in-app and email

Problem: users need to know when a borrow request is approved, or a book is overdue — even if they're not staring at the app.

Approach: `backend/src/services/scheduler.js` uses `node-cron` to run a job once a day (default `0 8 * * *` — 8 AM server time, overridable via the `NOTIFICATION_CRON` env var):

```
function startNotificationScheduler(prisma) {
  const schedule = process.env.NOTIFICATION_CRON || "0 8 * * *";
  cron.schedule(schedule, () => {
    notifyDueDates(prisma).catch(err => console.error("[scheduler] Failed:", err.message));
  });
}
```

`notificationService.js`'s `notifyDueDates` queries for borrows due in 3 days (sends a reminder email), and separately flips any `BORROWED` row whose `dueDate` has passed to `OVERDUE` and emails about that too. Email sending goes through `nodemailer`, configured from `SMTP_*` env vars — and **if `SMTP_HOST` isn't set, the transporter is simply `null` and every**

send is silently skipped, which is why the app works perfectly well in development with no mail server configured at all.

Separately, in-app notifications are just rows in the `Notification` table (Part 3.6), created by resolvers whenever a relevant event happens (a borrow request submitted, approved, rejected). The frontend's `NotificationsPage.tsx` queries them, and a bell icon in `Navigation.tsx` polls an unread count.

8.3 Image uploads via Cloudinary

Problem: profile pictures and book covers need to live *somewhere* — not as binary blobs in PostgreSQL.

Approach: the frontend reads a selected file, converts it to a base64 string, and sends that string as a GraphQL mutation argument. `backend/src/utils/cloudinary.js` strips the `data:image/...;base64,` prefix and hands the rest to Cloudinary's SDK, which uploads it and returns a permanent `secure_url` :

```
const result = await cloudinary.uploader.upload(
  `data:image/jpeg;base64,${imageData}`,
  { folder: "profile_pictures", transformation: [{ width: 500, height: 500, crop: "limit" }] }
);
// result.secure_url is what we store on the User row
```

Only that URL — a short string — ever touches the database. The resolver then does the access check you already recognize from §4.4 (`if (userId !== id && role !== "ADMIN") throw new Error("Not authorized")`) before saving it.

8.4 Soft deletes & audit logging

Problem: for a library system, "who deleted this book, and when" matters — and accidental deletes shouldn't be unrecoverable.

Approach: `User` and `Book` have a `deletedAt DateTime?` column (Part 3.6). "Deleting" one of these never actually runs SQL `DELETE` — it sets `deletedAt: new Date()` . Every query that lists these models filters `where: { deletedAt: null }` , and a separate admin-only query (`deletedUsers` , `deletedBooks`) lets an admin browse and restore them.

Separately, `backend/src/index.js` wraps the entire Prisma client in an **extension** that hooks every single database write:

```
const prisma = baseClient.$extends({
  query: {
    $allModels: {
      async $allOperations({ model, operation, args, query }) {
        const result = await query(args);
        if (AUDITED_MODELS.includes(model) && AUDITED_OPS[operation]) {
          baseClient.auditLog.create({
            data: { model, action: AUDITED_OPS[operation], recordId: result?.id, userId: /* from */
          }).catch(() => {});
        }
        return result;
      },
    },
  },
});
```

This means **no individual resolver has to remember to log anything** — every `create` / `update` / `delete` on an audited model is logged automatically, at the Prisma layer, once, for the whole app. This is the same principle as §6.1's "decide it on day one" — bolting audit logging onto 30 already-written resolvers individually would be far more error-prone than wrapping the one shared client.

Part 9 — Multi-Platform Builds (Survey)

The entire point of this chapter: **the React app you built in Part 5 does not change at all**. Only the *shell* around it changes.

9.1 Electron (Windows / Linux desktop)

Electron bundles a Chromium browser and a Node.js runtime together, and lets you run your web app as if it were a native desktop program. `frontend/electron/main.cjs` registers a custom `libroware://` protocol and serves your built `dist/` folder through it instead of `http://` / `https://` — this avoids various browser security restrictions that apply differently to `file://` URLs.

The one tricky part: your app's API URL logic (`frontend/src/config/api.ts`) needs to know it's running inside Electron *before React even renders*, since Apollo's `httpLink` is configured at module load (§5.2). A `preload.cjs` script bridges this by synchronously reading settings via Electron's IPC and exposing `window.electronAPI.getApiUrl()` to the page:

```
// frontend/src/config/api.ts
if (typeof window !== "undefined" && (window as any).electronAPI?.getApiUrl) {
  return (window as any).electronAPI.getApiUrl() as string;
}
```

`electron-builder` (configured in `frontend/package.json`'s `"build"` block) then packages `dist/` + `main.cjs` + `preload.cjs` into a signed `.exe` (NSIS installer) or `.deb` / `.AppImage`.

9.2 Capacitor (Android / iOS)

Capacitor takes the same built `dist/` folder and wraps it in a native WebView shell instead.

`frontend/capacitor.config.ts`:

```
const config: CapacitorConfig = {
  appId: "com.libroware.app",
  appName: "Libroware",
  webDir: "dist",
  server: { androidScheme: "https" }, // WebViews need https-like origin for cookies/storage
};
```

`npx cap sync` copies your latest `dist/` build into the native Android/iOS project folders. Because a mobile WebView has no Nginx reverse proxy sitting in front of it (unlike the web deployment in Part 10.2), the relative `/api/graphql` trick doesn't work — `api.ts` falls back to a hardcoded absolute `REMOTE_URL` in that case.

9.3 Why this works at all

Every platform-specific quirk funnels through exactly **one** function: `getApiUrl()` in `frontend/src/config/api.ts`. Everything else — every component, every GraphQL query, every piece of business logic — is completely unaware of whether it's running in a browser tab, an Electron window, or an Android WebView. This is the payoff of the "one React codebase" decision from Part 0: platform differences get isolated to the *smallest possible surface area*, instead of leaking into application code.

Part 10 — Containerization & Deployment

This is where your rebuilt app stops being "something that only runs on my laptop" and becomes a real, running service.

10.1 Why Docker

Without containers, "deploying" means manually installing the right Node version, the right PostgreSQL version, setting environment variables by hand, and hoping the production server's configuration matches your laptop's. **Docker** packages an application *and* everything it needs to run (the exact OS libraries, runtime version, dependencies) into an **image** — a single artifact that runs identically anywhere Docker is installed. A running instance of an image is a **container**.

10.2 Dockerizing the backend

The real `backend/Dockerfile` :

```
FROM node:20-slim

RUN apt-get update -y && apt-get install -y openssl

WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
RUN npx prisma generate

EXPOSE 4000
CMD ["npm", "start"]
```

Read this top to bottom — it's a *recipe*, executed once at build time: - `FROM node:20-slim` — start from a minimal pre-built image that already has Node 20. - `apt-get install openssl` — Prisma's query engine needs OpenSSL present on the OS to run. - `COPY package.json ./` then `RUN npm install` *before* `COPY . .` — this ordering is deliberate: Docker caches each instruction as a layer, and re-runs a layer only if its inputs changed. By copying just `package.json` first, `npm install` (the slow step) only re-runs when your *dependencies* change, not every time you edit a line of source code — dramatically speeding up repeated builds during development. - `RUN npx prisma generate` — regenerates the typed Prisma Client *inside the image*, so it matches the schema that was copied in. - `CMD ["npm", "start"]` — the command that runs when a container *starts* (as opposed to `RUN`, which only executes during the build).

10.3 Dockerizing the frontend — a multi-stage build

The real `frontend/Dockerfile` :

```
FROM node:20-slim AS build
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
RUN npx vite build

FROM nginx:stable-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

This is a **multi-stage build** — two `FROM` lines define two separate, named stages. The first stage (`AS build`) has the full Node.js toolchain and runs `vite build` , which compiles all your TypeScript/React/Tailwind into a handful of plain `.html` / `.css` / `.js` files in `dist/` . The second stage starts completely fresh from a tiny Nginx image and uses `COPY --from=build` to pull *only* the compiled `dist/` output across — discarding Node.js, `node_modules/` , and your source code entirely from the final image. The result: a production image that's a static file server, a few megabytes, with zero of your build tooling inside it. This pattern — build with heavy tools, ship with light ones — is standard practice for any compiled frontend.

The Nginx config it copies in (`frontend/nginx.conf`) does the other crucial job: it serves the static React app *and* reverse-proxies any request to `/api/graphql` over to the backend container, so the browser only ever talks to one origin:

```
location / {
    try_files $uri $uri/ /index.html; # React Router needs every path to fall back to index.html
}

location /api/graphql {
    proxy_pass http://backend:5000/graphql;
}
```

`try_files $uri $uri/ /index.html;` is the single most important line for any client-side-routed React app: without it, refreshing the browser on `/admin` would 404, because there's no real file at `/admin` on disk — only `index.html` , which React Router then takes over and renders the right screen for, *client-side*, once it loads.

10.4 Wiring it together with Docker Compose

A real app is more than one container — it's a backend, a frontend, and a database, all talking to each other. `docker-compose.yml` describes that whole group declaratively:

```

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      retries: 5

  backend:
    build: { context: ./backend }
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      - DATABASE_URL=${DATABASE_URL}
      - JWT_SECRET=${JWT_SECRET}
    command: sh -c "npx prisma migrate deploy && npm start"

  frontend:
    build: { context: ./frontend }
    depends_on: [backend]
    ports:
      - "3030:80"

volumes:
  postgres-data:

```

A few details that matter: - `depends_on: { postgres: { condition: service_healthy } }` — Compose won't even *start* the backend container until Postgres's `healthcheck` passes. Without this, the backend would race to connect to a database that hasn't finished initializing yet, and crash on startup. - `${DATABASE_URL}` — these are environment variable substitutions, read from a `.env` file sitting next to `docker-compose.yml` (Part 10.5). Nothing secret is hardcoded into the committed file. - `volumes: postgres-data:/var/lib/postgresql/data` — without this, every time you ran `docker compose down` (which removes containers), your *entire database* would vanish with it, because a container's own filesystem is disposable by design. A named volume is Docker's mechanism for data that should outlive any single container. - `command: sh -c "npx prisma migrate deploy && npm start"` — note this *overrides* the Dockerfile's `CMD`. Every time this container starts, it first applies any pending migrations (Part 3.5's tool, but `migrate deploy` instead of `migrate dev` — deploy *only* applies already-generated migrations, it never tries to generate new ones interactively, which is exactly what an unattended server needs), *then* starts the actual server. - `ports: "3030:80"` — only the frontend (Nginx) is exposed to the host machine's network; `backend` and `postgres` are reachable *only* from other

containers on the same Compose network, by their service name (`backend:5000` , as seen in `nginx.conf` above) — not from outside. This is a real security boundary, not just convenience.

✂ **Exercise 10.1** Write Dockerfiles for your own backend and frontend, and a `docker-compose.yml` joining them with a Postgres service, following the structure above. Create a `.env` next to it with `POSTGRES_USER` , `POSTGRES_PASSWORD` , `POSTGRES_DB` , `DATABASE_URL` (pointing at the `postgres` service name, not `localhost` , since containers address each other by service name), and `JWT_SECRET` . Run `docker compose up --build` and confirm all three containers start, and that your app is reachable in the browser at `http://localhost:3030` .

10.5 Environment variables and secrets

Notice that **nothing sensitive is ever committed to Git** — every secret (`DATABASE_URL` with its password, `JWT_SECRET` , `CLOUDINARY_URL`) lives in a `.env` file that's listed in `.gitignore` , and is read at runtime via `process.env.X` (backend) or build-time substitution (Compose). For a real deployed server, that `.env` has to get onto the server somehow *without* ever touching Git history — Part 10.6 covers exactly how Libroware does this, via a GitHub Actions secret.

10.6 CI/CD: deploying with GitHub Actions

CI/CD (Continuous Integration / Continuous Deployment) means: instead of manually SSH-ing into a server and running commands by hand every time you want to ship a change, a script — triggered automatically or on demand — does it identically, every time. Libroware's real deploy workflow, `.github/workflows/libroware_deploy.yml` , triggers **manually** (`workflow_dispatch` — a button you click in GitHub's UI, not on every `git push` , which is a deliberate safety choice for a small team):

```

on:
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Connect to Tailscale
        uses: tailscale/github-action@v2
        with:
          oauth-client-id: ${ secrets.TS_OAUTH_CLIENT_ID }
          oauth-secret: ${ secrets.TS_OAUTH_CLIENT_SECRET }

      - name: Clone repo on server
        run: |
          ssh ${ secrets.SERVER_USER }@${ secrets.SERVER_HOST } "
            cd /home/${ secrets.SERVER_USER }
            rm -rf libroware
            git clone https://github.com/KrisEllaDavid/libroware.git
          "

      - name: Write environment file
        env:
          ENV_FILE: ${ secrets.ENV_FILE }
        run: |
          echo "$ENV_FILE" | ssh ... "cat > /home/.../libroware/.env"

      - name: Start application
        run: |
          ssh ... "cd /home/.../libroware && ./deploy.sh"

```

Each `${ secrets.X }` is a value stored in **GitHub's encrypted secrets store** (Settings → Secrets and variables → Actions on the repo) — never visible in logs, never committed anywhere. The workflow: connects to the deploy target over a private network (Tailscale — a VPN mesh, used here so the server doesn't need a public SSH port exposed to the entire internet), wipes and re-clones the repo fresh on the server, writes the `.env` file's *entire contents* from a single `ENV_FILE` secret, then runs `deploy.sh` — which is just `docker compose up --build -d` plus a couple of housekeeping steps. This is the entirety of "deployment" for this project: clone, write config, `docker compose up`.

10.7 Going live with a domain

Right now the app is reachable by raw IP and port (`http://185.217.125.37:3030`).

`SERVER_CONFIG.md` (this repo, §7) documents the remaining steps once you own a domain:

1. **DNS** — point an A record at the server's IP.

2. **A second, system-level Nginx** (outside Docker, installed directly on the server) terminates HTTPS and reverse-proxies to the Dockerized frontend's exposed port: `nginx`

```
server { listen 443 ssl; server_name yourdomain.com; ssl_certificate  
/etc/letsencrypt/live/yourdomain.com/fullchain.pem; ssl_certificate_key  
/etc/letsencrypt/live/yourdomain.com/privkey.pem; location / { proxy_pass  
http://127.0.0.1:3030; } }
```
3. `certbot --nginx -d yourdomain.com` — Let's Encrypt issues a free TLS certificate and certbot wires it into that config automatically.
4. **Update `ALLOWED_ORIGINS`** (§4.2's CORS check) and the frontend's hardcoded `REMOTE_URL` (§9.2/9.3) to the new `https://` domain, then redeploy.

Two Nginxes are involved on purpose: the one *inside* the frontend Docker image (§10.3) only ever needs to know about plain HTTP and the backend container's internal address; the system Nginx is the only thing that needs to know about certificates and the public domain. Each stays simple by not needing to know about the other's job.

10.8 Post-deploy database migrations

Notice the backend container's `command` in §10.4 already runs `prisma migrate deploy` on every start — so for *most* schema changes, a normal redeploy handles migrations automatically. `SERVER_CONFIG.md` documents the one-off manual form for cases where you need to run it by hand without a full redeploy:

```
ssh <user>@<server>  
cd ~/libroware  
docker compose exec backend npx prisma migrate deploy
```

`docker compose exec backend ...` runs a command *inside* the already-running `backend` container — useful for one-off maintenance without restarting anything.

✂ **Exercise 10.2** Write your own `deploy.sh` (a handful of lines: `git pull`, `docker compose up --build -d`, maybe a migration step) and your own minimal GitHub Actions workflow file under `.github/workflows/` with `on: workflow_dispatch`. You don't need a real server to do this exercise meaningfully — write the workflow, understand what each step would do, and if you have access to *any* SSH-reachable machine (even a free-tier cloud VM), try running it for real.

Part 11 — Capstone Checklist & Where to Go Next

11.1 Full rebuild checklist

If you completed every exercise, you now have, in your own words and your own files:

- [] A Postgres database running in Docker, modeled with Prisma (Part 3)
- [] A GraphQL API with schema-first types, JWT authentication via httpOnly cookies, and role-based authorization on every mutation (Part 4)
- [] A React + TypeScript frontend with Apollo Client, a global auth context gating the entire route tree, and at least two full CRUD admin screens (Part 5)
- [] That same frontend rendering correctly in two languages via `react-i18next` (Part 6)
- [] A working fix for the "expired session leaves the UI dead" bug, using a `window` event bridge (Part 7)
- [] A clear mental model of offline queuing, scheduled email notifications, image uploads, and audit logging — even without having rebuilt all of them (Part 8)
- [] An understanding of why Electron and Capacitor need no changes to your React code, only a different shell (Part 9)
- [] Dockerfiles for both halves of the app, a `docker-compose.yml` joining them with Postgres, and a manually-triggered GitHub Actions deploy workflow (Part 10)

11.2 Stretch goals

Once the above is solid, here are natural next features — each one is "apply a pattern you already know, to a new entity":

- **Borrow & Fine workflow:** add the `Borrow` and `Fine` models from Part 3.6 yourself, with a resolver that calculates a fine automatically based on `daysOverdue * dailyRate` when a book is returned late.
- **Reservations:** let a user join a queue for a currently-unavailable book, and automatically notify the next person in line when it's returned.
- **Reviews:** a one-to-many `Review` on `Book`, with a rating constrained between 1 and 5 (validate this in the resolver, the same way `signup` validates password length).
- **Tests:** this course never wrote a single automated test. Pick one resolver (start with `author.js`) and write a test for it using Jest, mocking the `prisma` client — this is the single highest-leverage thing you could add to harden everything you've built.

11.3 Further learning

- **Prisma docs** (prisma.io/docs) — the relations, migrations, and filtering API go far deeper than what's used here.
 - **Apollo Client docs** (apollographql.com/docs/react) — caching strategies, pagination patterns, subscriptions (real-time GraphQL, not used in Libroware at all today).
 - **react-i18next docs** — pluralization rules, namespacing, lazy-loading translations for larger apps.
 - **Docker's own "Get Started" guide** — multi-stage builds, build caching, and `docker compose` profiles go deeper than what we covered.
-

This course was built directly from the real, running Libroware codebase — every code block in Parts 3–10 is either an exact excerpt or a deliberately trimmed version of the actual file referenced. Where you see a path like `backend/src/graphql/resolvers/author.js`, that file exists, in full, in the project repository — read it after finishing each chapter's exercise to compare your own solution against the original.